The Complete & Advanced React Interview Cheatsheet

Engineered for Senior React Engineers & Technical Interviews. Master low-level Fiber internals, reconciliation rules, hook memory structures, concurrent mode features, advanced design patterns, and high-frequency interview edge cases.

- ⚡ 10 Deep Modules
- 🚀 React 18 & 19 Ready
- 🎯 50+ Senior Interview Insights
- 💡 Practical Production Snippets

1. Core Architecture & Virtual DOM CORE

Virtual DOM & Reconciliation Heuristics

React creates a lightweight in-memory tree of JS objects representing the UI. Reconciliation is the algorithm React uses to diff the current tree with a new virtual tree to compute minimal real DOM operations.

> **Senior Interview Answer:** React's diffing runs in **O(n)** time based on two key heuristics:
> 1. *Different element types* produce different trees (React tears down the old tree and builds a new one from scratch).
> 2. *Keys* hint to React which child elements remain stable across re-renders.

reconciliation.js

```
// React Element representation (JS Object)
const element = {
  $$typeof: Symbol.for('react.element'), // Security protection against XSS via JSON
  type: 'button',
  props: { className: 'btn', children: 'Click Me' },
  key: null,
  ref: null
};
```

- ✓ Elements are immutable blue-print objects; Fibers are mutable units of work.
- ✓ `$$typeof: Symbol.for('react.element')` prevents raw JSON injected from APIs from being rendered as React DOM nodes.

Keying Strategies & Re-mounting Pitfalls

Keys must be stable, unique, and predictable. Using indexes as keys when order can change breaks component internal state and degrades performance.

> **Beware:** Using `key={Math.random()}` forces React to unmount and remount the node on *every single render*, destroying state and firing full DOM layout recalcs!

keys-comparison.jsx

```jsx
// ❌ BAD: Index key breaks state when items are reordered/prepended
{items.map((item, index) => <ItemCard key={index} data={item} />)}

// ❌ WORST: Random key destroys subtree on every render
<ItemCard key={Math.random()} />

// ✅ GOOD: Stable entity ID
{items.map(item => <ItemCard key={item.id} data={item} />)}
```

- ✓ Changing `key` on an element intentionally resets component state (useful technique for resetting forms).
- ✓ Keys are evaluated strictly among sibling nodes, not globally.

2. Component Lifecycles & Error Boundaries LIFECYCLES

Class Lifecycles vs Hooks Mapping

Functional components do not have lifecycles per se; they synchronize side effects with state and props via `useEffect` / `useLayoutEffect`.

lifecycle-mapping.js

```
/*
  Class Method              ⟶ Hook Equivalent
  ═══════════════════════   ═══════════════════════════════════════
  componentDidMount         ⟶ useEffect(() ⇒ {}, [])
  componentDidUpdate        ⟶ useEffect(() ⇒ {}, [deps])
  componentWillUnmount      ⟶ useEffect(() ⇒ { return () ⇒ cleanup }, [])
  shouldComponentUpdate     ⟶ React.memo(Comp, prevProps ⩵ nextProps)
  getDerivedStateFromProps  ⟶ Update state during render (if required)
  getSnapshotBeforeUpdate   ⟶ useLayoutEffect (before paint read DOM)
*/
```

> **Interview Question:** When does `useEffect` cleanup run?
> Answer: The cleanup function runs **before** the component unmounts AND **before re-running the effect** on dependency changes (to clean up the previous render's side effect).

Error Boundaries (Class Only)

Error boundaries catch JavaScript errors anywhere in child component trees, log those errors, and display a fallback UI instead of crashing the whole component tree.

Crucial Interview Catch: Error Boundaries do **NOT** catch errors inside:
1. Event Handlers (use try/catch inside handler instead)
2. Asynchronous code (`setTimeout`, `fetch`, Promises)
3. Server-side rendering (SSR)
4. Errors thrown in the boundary itself.

ErrorBoundary.jsx

```jsx
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error) {
    // Update state so the next render shows fallback UI
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    // Log error to reporting services (Sentry, PostHog, Datadog)
    logErrorToService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback || <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}
```

3. Hooks Deep Dive & Fiber Linked List HOOKS MECHANICS

How Hooks Work Internally (Linked List)

Hooks do NOT rely on magic; they rely on array/linked-list indexes attached to the Fiber node (`fiber.memoizedState`).

> **Why Rules of Hooks exist:** React maintains a global `currentlyRenderingFiber` pointer and a `workInProgressHook` index. If you call hooks inside `if` statements or loops, the order of calls shifts between renders, pointing to the wrong hook node in the linked list!

simplified-hooks-engine.js

```
// Conceptual mental model of React Hooks linked list
let workInProgressHook = null;

function useWorkInProgressHook() {
  if (!workInProgressHook) {
    // Mount phase: create new hook node
    workInProgressHook = { memoizedState: null, next: null };
  } else {
    // Update phase: advance pointer in linked list
    workInProgressHook = workInProgressHook.next;
  }
  return workInProgressHook;
}
```

useEffect vs useLayoutEffect vs useInsertionEffect

Understanding execution timing relative to DOM mutations, layout calculation, and browser painting.

effect-execution-order.js

```
/*
  1. React Renders Virtual DOM
  2. useInsertionEffect fires (DOM mutates, styles inserted; before layout calc)
  3. React Mutates DOM Nodes synchronously
  4. useLayoutEffect fires SYNCHRONOUSLY before browser paints (good for scroll measurement)
  5. Browser Paints the pixels on screen
  6. useEffect fires ASYNCHRONOUSLY (non-blocking for performance)
*/
```

> **Rule of Thumb:** Use `useEffect` 99% of the time. Use `useLayoutEffect` only when reading/measuring DOM dimensions or preventing visual flicker during immediate state transitions.

useState: Functional Updates & Lazy Init

State updates are batched. Passing a function to state setters guarantees evaluation against the freshest pending state.

state-best-practices.jsx

```jsx
// ✅ Lazy Initial State: fn runs ONLY ONCE during mount, NOT on re-renders
const [config, setConfig] = useState(() => heavyExpensiveComputation());

// ✅ Functional Update: Prevents stale closure issues when queuing updates
const increment = () => {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1); // Correctly increments by +2
};
```

useSyncExternalStore (React 18+)

Recommended hook for subscribing to external state stores (Zustand, Redux, browser APIs) without concurrent tearing issues.

useWindowWidth.js

```jsx
function subscribe(callback) {
  window.addEventListener('resize', callback);
  return () => window.removeEventListener('resize', callback);
}

function useWindowWidth() {
  return useSyncExternalStore(
    subscribe,
    () => window.innerWidth,    // Client snapshot
    () => 1024                  // SSR initial snapshot
  );
}
```

4. React 18 & React 19 Modern Features REACT 18/19

React 19: The `use()` Hook

The `use()` API allows reading promises or context conditionally inside render. Unlike standard hooks, `use()` CAN be called inside conditionals and loops!

React19UseHook.jsx

```jsx
import { use, Suspense } from 'react';

function UserProfile({ userPromise, showDetails }) {
  // ✅ Conditional call allowed for use()
  if (!showDetails) return <p>Hidden</p>;

  // Unwraps promise; suspends component until resolved
  const user = use(userPromise);
  return <div>{user.name}</div>;
}
```

Concurrent React: useTransition & useDeferredValue

Allows marking state updates as low-priority transitions. Keystrokes/clicks remain instant while expensive non-urgent renders yield to user input.

useTransitionExample.jsx

```jsx
const [isPending, startTransition] = useTransition();
const [filter, setFilter] = useState('');

const handleChange = (e) => {
  // High priority: Input text updates immediately
  const val = e.target.value;

  // Low priority: Filtering 10,000 items is interruptible
  startTransition(() => {
    setFilter(val);
  });
};
```

React 19: Actions, useActionState & useFormStatus

First-class support for async form actions, optimistic state resets, pending indicators, and server actions.

FormActionExample.jsx

```jsx
// React 19 Form Action state management
const [state, formAction, isPending] = useActionState(
  async (prevState, formData) => {
    const res = await updateUser(formData.get('name'));
    return res.success ? { error: null } : { error: res.msg };
  },
  { error: null }
);

return (
  <form action={formAction}>
    <input name="name" />
    <button disabled={isPending}>Submit</button>
  </form>
);
```

React Server Components (RSC) vs Client Components

Server Components run strictly on the server during render, stream zero JS bundle to the client, and can access databases directly.

> **Key Difference:** Client components (`"use client"`) render on the server initially (SSR) AND re-render on the client. Server components NEVER execute on the client browser runtime.

5. Advanced React Design Patterns PATTERNS

Compound Components Pattern

Allows components to share implicit state while giving consumers complete UI rendering freedom (e.g. Radix UI, Headless UI, Select, Accordion).

CompoundSelect.jsx

```jsx
const SelectContext = createContext();

function Select({ children, value, onChange }) {
  return (
    <SelectContext.Provider value={{ value, onChange }}>
      <div className="select-box">{children}</div>
    </SelectContext.Provider>
  );
}

Select.Option = function Option({ value, children }) {
  const { value: selectedValue, onChange } = useContext(SelectContext);
  const isSelected = selectedValue === value;
  return (
    <div onClick={() => onChange(value)} className={isSelected ? 'active' : ''}>
      {children}
    </div>
  );
};

// Consumer Usage:
<Select value={val} onChange={setVal}>
  <Select.Option value="react">React</Select.Option>
  <Select.Option value="vue">Vue</Select.Option>
</Select>
```

Higher-Order Components (HOC) & Static Hoisting

A function that takes a component and returns an enhanced component. Remember to forward refs and hoist static methods.

withAuth.jsx

```
function withAuth(WrappedComponent) {
  const WithAuthComponent = forwardRef((props, ref) => {
    const { isAuthenticated } = useAuth();
    if (!isAuthenticated) return <Redirect to="/login" />;
    return <WrappedComponent {...props} ref={ref} />;
  });

  WithAuthComponent.displayName = `withAuth(${WrappedComponent.displayName || WrappedComponent.name})`;
  return WithAuthComponent;
}
```

6. Performance Optimization Blueprint PERFORMANCE

Composition over Memoization (Moving State Down)

Before reaching for `useMemo` or `React.memo`, restructure your component tree. Passing expensive subtrees via `children` avoids unnecessary re-renders naturally.

composition-perf.jsx

```jsx
// ✅ Extremely Performant: HeavyComponent DOES NOT re-render on scroll!
function ScrollContainer({ children }) {
  const [scrollPos, setScrollPos] = useState(0);
  return (
    <div onScroll={e => setScrollPos(e.target.scrollTop)}>
      <p>Pos: {scrollPos}</p>
      {children} /* HeavyComponent passed as prop remains unchanged object */
    </div>
  );
}
```

React.memo & Custom Comparator

`React.memo` performs shallow comparison of props by default (`Object.is`).

MemoCustomCompare.jsx

```jsx
const ExpensiveCard = React.memo(
  function Card({ item }) {
    return <div>{item.title}</div>;
  },
  (prevProps, nextProps) => {
    // Return TRUE if props are equal (DO NOT re-render)
    return prevProps.item.id === nextProps.item.id;
  }
);
```

7. State Management Matrix STATE ARCHITECTURE

State Classification Matrix

Choosing the right tool for the state domain is critical for maintainability and avoiding context re-render waterfalls.

- **Local UI State:** `useState`, `useReducer`
- **Global UI Theme/Auth:** React Context API
- **Global Complex App State:** Zustand, Redux Toolkit, Jotai
- **Server/API State:** TanStack Query (React Query), SWR (handles caching, deduping, revalidation)

8. Gotchas, Pitfalls & Beware List (Interview Critical) PITFALLS

Stale Closures in useEffect & Handlers

A function closed over variables from an old render snapshot retains those outdated references unless re-created or accessed via ref.

stale-closure-fix.jsx

```jsx
// ❌ BUGGY: count is captured at mount time (= 0); interval logs 1 forever!
useEffect(() => {
  const id = setInterval(() => {
    setCount(count + 1);
  }, 1000);
  return () => clearInterval(id);
}, []);

// ✅ FIX 1: Functional state update (doesn't depend on count variable)
setCount(c => c + 1);

// ✅ FIX 2: useRef for latest callback pattern (useLatest)
const countRef = useRef(count);
countRef.current = count;
```

Infinite Re-render Loops

Occurs when setting state directly in the component body, or passing un-memoized object/array literals into effect dependency arrays.

infinite-loop-traps.jsx

```jsx
// ❌ CAUSES INFINITE LOOP: New object reference created every render!
const options = { page: 1 };
useEffect(() => {
  fetchData(options);
}, [options]); // Options fails shallow equality check every time

// ✅ FIX: Primitive dependencies or useMemo
useEffect(() => {
  fetchData({ page });
}, [page]);
```

9. Fiber Architecture & Reconciler Internals FIBER INTERNALS

The Fiber Node Data Structure

A Fiber is a JavaScript object representing a unit of work. Prior to Fiber (React 16), reconciliation was recursive and un-interruptible (Stack Reconciler). Fiber turned the call stack into a linked list structure operating on a virtual stack frame.

FiberNode.ts

```
interface FiberNode {
  tag: WorkTag;           // FunctionComponent, ClassComponent, HostComponent (DOM node)
  key: null | string;
  elementType: any;
  type: any;

  // Tree Structure Links
  return: FiberNode | null;    // Parent fiber
  child: FiberNode | null;     // First child fiber
  sibling: FiberNode | null;   // Next sibling fiber

  // Hooks & State
  memoizedState: any;        // Linked list of hooks for functional components
  memoizedProps: any;

  // Double Buffering Mechanism
  alternate: FiberNode | null;// Points to work-in-progress fiber or current fiber

  // Effects & Commit Flags
  flags: Flags;               // Insertion, Update, Deletion DOM effect tags
}
```

Double Buffering Strategy: React maintains two Fiber trees at all times:
1. **Current Tree:** Represents what is currently painted on screen.
2. **Work-In-Progress (WIP) Tree:** Constructed asynchronously in memory during the render phase. Once rendered cleanly, React flips the single root pointer `root.current = workInProgress`, making the WIP tree current instantaneously!

10. Tricky Interview Code Snippets & Custom Hooks

CODING CHALLENGES

Custom Hook: useDebounce

Implement a hook that delays updating a value until a specified timeout has elapsed since the last change.

useDebounce.js

```
function useDebounce(value, delay = 300) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

Custom Hook: usePrevious

Tracks the value of a prop or state from the immediately preceding render cycle using `useRef` and `useEffect`.

usePrevious.js

```
function usePrevious(value) {
  const ref = useRef();

  useEffect(() => {
    ref.current = value; // Updates AFTER render phase completes
  }, [value]);

  return ref.current; // Returns value held DURING render phase
}
```